When the docker build command is run, the CLI tool takes all the files in the build directory specified—which collectively are known as the *build context*—and sends them to the Docker daemon. The daemon is then able to process the Dockerfile and carry out the instructions in it to generate the image.

We need a way to restrict which files are sent as part of the build context because sending more files slows down your builds (especially true on Docker for Mac or Windows where the daemon is running in a virtualized host). Also, we might want to prevent sensitive files containing secrets from being included in our image—particularly if we plan to share the image publicly.

To exclude certain files and directories from being sent as part of the build context, we list them in a .dockerignore file in our build directory. The .dockerignore file works on a similar basis to a .gitignore file, which you're probably familiar with, although the pattern matching syntax is slightly different.[2]

Let's create a basic .dockerignore file for our project:

```
Line 1  # Git
      - .git
      - .gitignore
      -
      5 # Logs
      - log/*
      -
      - # Temp files
      - tmp/*
     10
      - # Editor temp files
      - *.swp
      - *.swo
```

We exclude the .git directory (line 2), which contains the Git history and config, as our image only needs the latest version of the files. Although a minor thing, while we're at it, we've excluded the .gitignore file too (line 3).

Similarly, we exclude any logs (line 6) or temp files (9), as these are generated and can safely be ignored. Finally, I'm excluding Vim's temporary .swp and .swo files (lines 12 and 13)—feel free to do the same for your editor of choice.

This .dockerignore file is a good starting place, but you can really go to town and ignore all cached or generated files.

Let's rebuild our image with the .dockerignore file in place.

2. https://docs.docker.com/engine/reference/builder/#dockerignore-file